

Student Performance & Attendance Analysis

Problem statement: Schools collect attendance, homework, and exam data separately, making it hard to see how these factors relate to each other at a glance. This notebook cleans, merges, and analyzes those datasets to surface subject-level performance trends, attendance patterns, and the relationship (if any) between attendance/homework habits and exam scores.

Dataset: [Student Performance and Attendance Dataset](#) by **Marvy Ayman Halim**, via Kaggle. Five CSV files: students, attendance, homework, performance (exam scores), and teacher-parent communication logs.

Methods: pandas for cleaning/merging/aggregation, matplotlib and Plotly for visualization. See [DATA_CLEANING_BRIEF.md](#) in `../data/` for a full writeup of cleaning decisions.

1. Imports & Setup

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import plotly.express as px
from pathlib import Path
from IPython.display import HTML, display

pd.set_option("display.max_columns", None)
DATA_DIR = Path("../dashboard_app/data")
```

2. Load Raw Data

Loading all five source files as-is, before any cleaning.

```
In [2]: students_raw = pd.read_csv(DATA_DIR / "students.csv")
attendance_raw = pd.read_csv(DATA_DIR / "attendance.csv")
homework_raw = pd.read_csv(DATA_DIR / "homework.csv")
performance_raw = pd.read_csv(DATA_DIR / "performance.csv")
communication_raw = pd.read_csv(DATA_DIR / "teacher_parent_communication.csv")

for name, df in [("students", students_raw), ("attendance", attendance_raw),
                ("homework", homework_raw), ("performance", performance_raw),
                ("communication", communication_raw)]:
    print(f"{name}: {df.shape[0]:,} rows x {df.shape[1]} cols")
```

```
students: 12,156 rows x 5 cols
attendance: 364,680 rows x 4 cols
homework: 60,780 rows x 7 cols
performance: 36,468 rows x 5 cols
communication: 24,312 rows x 4 cols
```

3. Data Quality Issues Found

A quick look at raw values shows several inconsistencies that need to be resolved before the files can be merged or analyzed reliably.

```
In [3]: print("Raw attendance status values:")
print(attendance_raw["Attendance_Status"].unique())
print()
print("Raw homework status values:")
print(homework_raw["Status"].unique())
print()
print("Raw homework completion % samples:")
print(performance_raw["Homework_Completion_%"].unique()[:10])
print()
print("Raw date samples (attendance):")
print(attendance_raw["Date"].unique()[:10])
```

```
Raw attendance status values:
<ArrowStringArray>
[  'Present',  'PRESENT',  'excused',  'late',  'Absent',
  'left early',  'Late',  'absnt']
Length: 8, dtype: str
```

```
Raw homework status values:
<ArrowStringArray>
['x', 'Done ', '👍', '✓', 'not done', 'pending']
Length: 6, dtype: str
```

```
Raw homework completion % samples:
<ArrowStringArray>
['90', '100%', '95', '80%', '-5']
Length: 5, dtype: str
```

```
Raw date samples (attendance):
<ArrowStringArray>
['2024-09-20', '2025-01-22', '2024-12-03', '2024-03-16', '2024-05-24',
 '2024-03-13', '2024-07-10', '2024-08-06', '2024-12-11', '2024-03-25']
Length: 10, dtype: str
```

Issues identified:

- `Attendance_Status` mixes case and stray whitespace (e.g. `Present`, `PRESENT`, `excused`)
- `homework.Status` uses emoji (👍 / x) instead of readable text
- `Homework_Completion_%` mixes plain numbers and percent-signed strings (`90` vs `100%`)
- Dates appear in more than one format (`YYYY-MM-DD` and `DD-MM-YYYY`)
- `students.csv` has no gender field — grade-level is used as the class/grouping variable instead

4. Cleaning

```
In [4]: def parse_date(series):
        return pd.to_datetime(series, errors="coerce", format="mixed")

def clean_percent(series):
    s = series.astype(str).str.strip().str.replace("%", "", regex=False)
    return pd.to_numeric(s, errors="coerce")

# --- students ---
students = students_raw.copy()
students["Date_of_Birth"] = parse_date(students["Date_of_Birth"])
students["Grade_Level"] = students["Grade_Level"].astype(str).str.strip()

# --- attendance ---
attendance = attendance_raw.copy()
attendance["Date"] = parse_date(attendance["Date"])
status = attendance["Attendance_Status"].astype(str).str.strip().str.lower()
status_map = {"present": "Present", "absent": "Absent", "excused": "Excused", "late": "Late"}
attendance["Attendance_Status"] = status.map(status_map).fillna(status.str.title())
attendance["Subject"] = attendance["Subject"].astype(str).str.strip()

# --- homework ---
homework = homework_raw.copy()
homework["Due_Date"] = parse_date(homework["Due_Date"])
status = homework["Status"].astype(str).str.strip()
homework["Status"] = status.map({"x": "Not Done", "👍": "Done", "Done": "Done", "done": "Done"}).fillna(status)
homework["Subject"] = homework["Subject"].astype(str).str.strip()

# --- performance ---
performance = performance_raw.copy()
performance["Exam_Score"] = pd.to_numeric(performance["Exam_Score"], errors="coerce")
performance["Homework_Completion_%"] = clean_percent(performance["Homework_Completion_%"])
performance["Subject"] = performance["Subject"].astype(str).str.strip()

# --- communication ---
communication = communication_raw.copy()
communication["Date"] = parse_date(communication["Date"])

print("Cleaned attendance status values:", attendance["Attendance_Status"].unique())
print("Cleaned homework status values:", homework["Status"].unique())
```

```
Cleaned attendance status values: <ArrowStringArray>
['Present', 'Excused', 'Late', 'Absent', 'Left Early', 'Absnt']
Length: 6, dtype: str
Cleaned homework status values: <ArrowStringArray>
['Not Done', 'Done', '✓', 'not done', 'pending']
Length: 5, dtype: str
```

5. Merge into a Master Analysis Table

One row per (student, subject), combining exam performance with attendance rate and homework completion for that student.

```
In [5]: attendance["is_present"] = attendance["Attendance_Status"] == "Present"
att_overall = (attendance.groupby("Student_ID")["is_present"].mean().mul(100).round(1)
               .rename("Overall_Attendance_Rate_%").reset_index())
att_subject = (attendance.groupby(["Student_ID", "Subject"])["is_present"].mean().mul(100).round(1)
               .rename("Attendance_Rate_%").reset_index())

homework["is_done"] = homework["Status"] == "Done"
hw_overall = (homework.groupby("Student_ID")["is_done"].mean().mul(100).round(1)
              .rename("Homework_Done_Rate_%").reset_index())

perf_agg = (performance.groupby(["Student_ID", "Subject"])
            .agg(Exam_Score=("Exam_Score", "mean"),
                 Homework_Completion_pct=("Homework_Completion_%", "mean"))
            .reset_index())

master = perf_agg.merge(students[["Student_ID", "Full_Name", "Grade_Level"]], on="Student_ID", how="left")
master = master.merge(att_subject, on=["Student_ID", "Subject"], how="left")
master = master.merge(att_overall, on="Student_ID", how="left")
master = master.merge(hw_overall, on="Student_ID", how="left")

print(f"Master table: {master.shape[0]:,} rows x {master.shape[1]} cols")
master.head()
```

Master table: 28,736 rows x 9 cols

```
Out[5]:
```

	Student_ID	Subject	Exam_Score	Homework_Completion_pct	Full_Name	Grade_Level	Attendance_Rate_%	Overall_
0	S00002	Arabic	69.0	-5.0	John Stafford	Grade 5	28.6	
1	S00002	History	98.0	90.0	John Stafford	Grade 5	100.0	
2	S00003	History	63.0	-5.0	Chad Harper	Grade 3	25.0	
3	S00004	English	106.0	-5.0	Anthony Martin	Grade 5	25.0	
4	S00004	History	59.0	90.0	Anthony Martin	Grade 5	100.0	

6. Computed Indicators

Mean and variance of exam scores per subject, and correlation between exam score and (a) attendance rate, (b) homework completion.

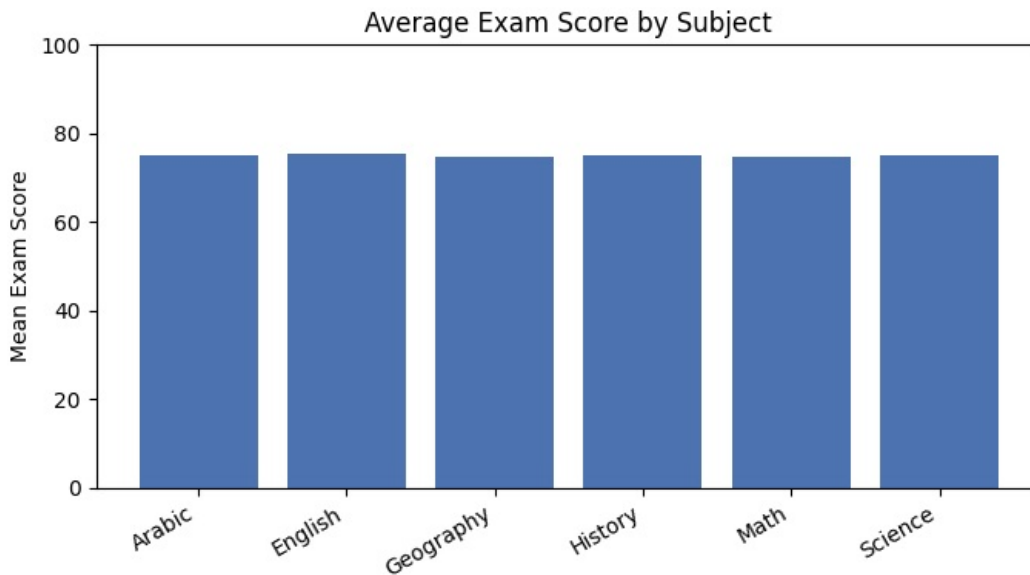
```
In [6]: rows = []
for subject, g in master.groupby("Subject"):
    rows.append({
        "Subject": subject,
        "Mean_Score": round(g["Exam_Score"].mean(), 2),
        "Variance": round(g["Exam_Score"].var(), 2),
        "Corr_Attendance_vs_Score": round(g["Attendance_Rate_%"].corr(g["Exam_Score"]), 2),
        "Corr_Homework_vs_Score": round(g["Homework_Completion_pct"].corr(g["Exam_Score"]), 2),
    })
indicators = pd.DataFrame(rows).sort_values("Subject").reset_index(drop=True)
indicators
```

Out[6]:	Subject	Mean_Score	Variance	Corr_Attendance_vs_Score	Corr_Homework_vs_Score
0	Arabic	74.97	374.47	-0.01	0.04
1	English	75.30	376.76	-0.00	0.02
2	Geography	74.57	366.56	-0.01	0.00
3	History	75.14	376.49	-0.02	0.01
4	Math	74.64	373.15	-0.02	-0.00
5	Science	75.13	364.80	0.01	-0.01

7. Visualizations

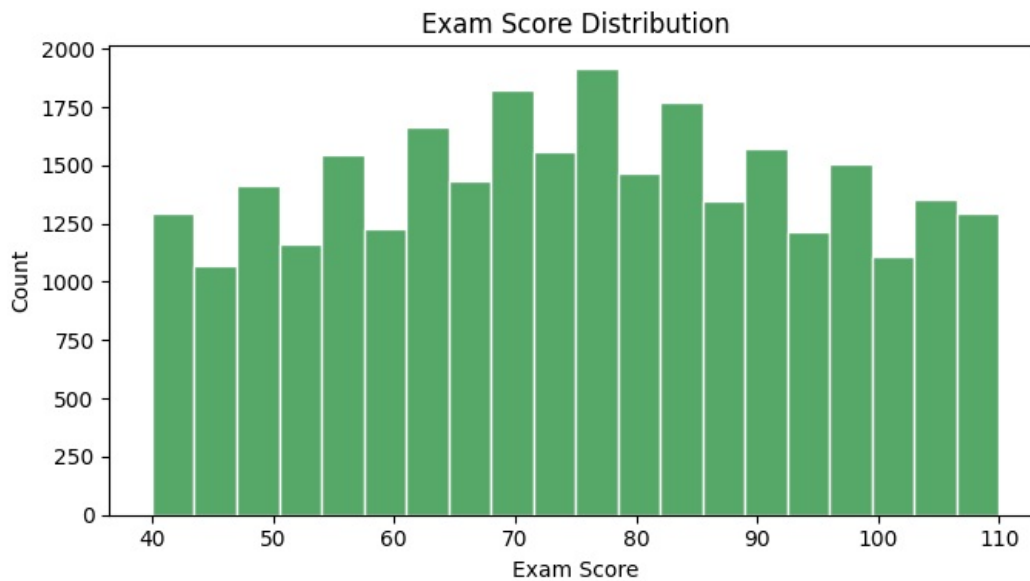
7.1 Average Exam Score by Subject

```
In [7]: fig, ax = plt.subplots(figsize=(7, 4))
ax.bar(indicators["Subject"], indicators["Mean_Score"], color="#4C72B0")
ax.set_ylabel("Mean Exam Score")
ax.set_ylim(0, 100)
ax.set_title("Average Exam Score by Subject")
plt.xticks(rotation=30, ha="right")
plt.tight_layout()
plt.show()
```



7.2 Exam Score Distribution

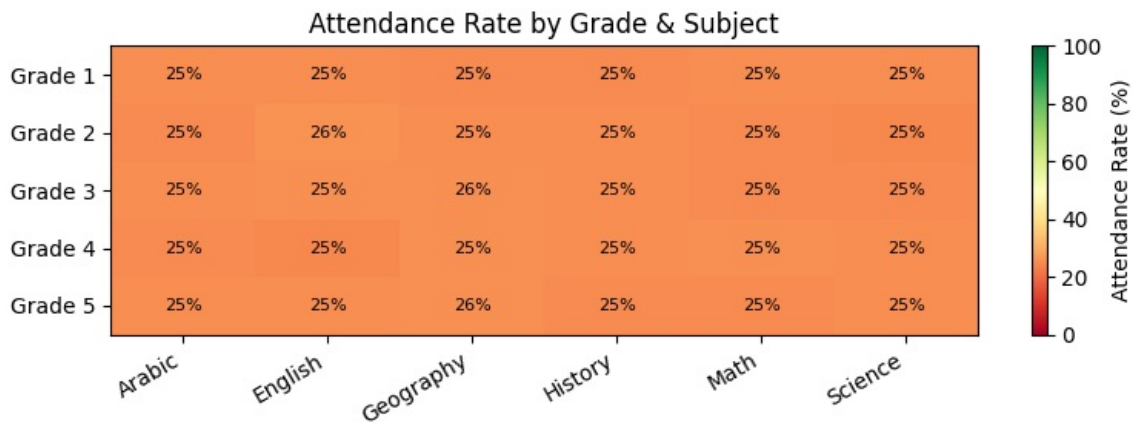
```
In [8]: fig, ax = plt.subplots(figsize=(7, 4))
ax.hist(master["Exam_Score"].dropna(), bins=20, color="#55A868", edgecolor="white")
ax.set_xlabel("Exam Score")
ax.set_ylabel("Count")
ax.set_title("Exam Score Distribution")
plt.tight_layout()
plt.show()
```



7.3 Attendance Rate: Grade vs. Subject

```
In [9]: att_students = attendance.merge(students[["Student_ID", "Grade_Level"]], on="Student_ID", how="left")
heatmap_data = att_students.pivot_table(index="Grade_Level", columns="Subject", values="is_present", aggfunc="mean")

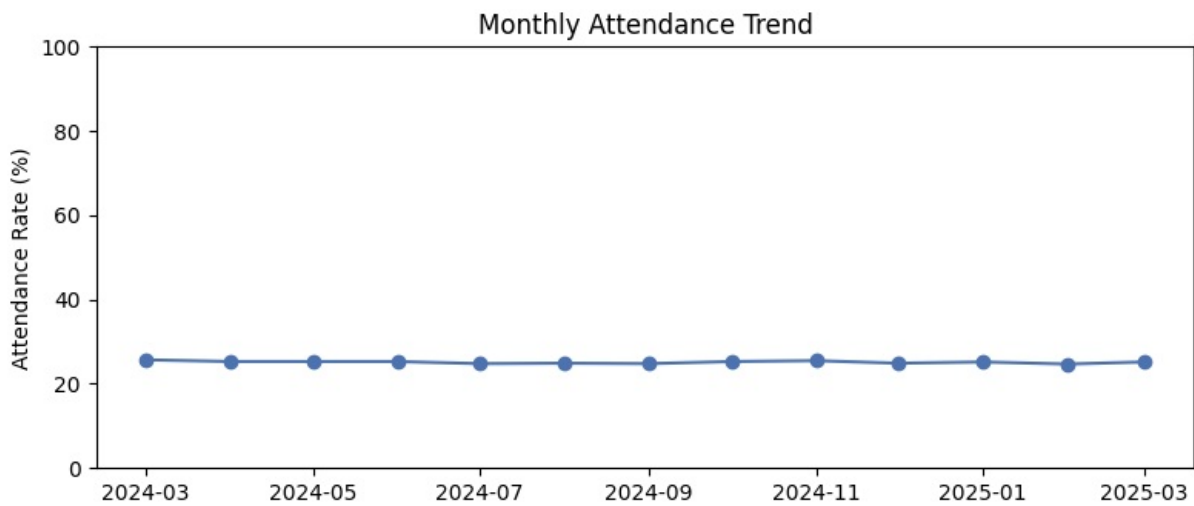
fig, ax = plt.subplots(figsize=(8, max(3, 0.5 * len(heatmap_data))))
im = ax.imshow(heatmap_data.values, cmap="RdYlGn", vmin=0, vmax=100, aspect="auto")
ax.set_xticks(range(len(heatmap_data.columns))); ax.set_xticklabels(heatmap_data.columns, rotation=30, ha="right")
ax.set_yticks(range(len(heatmap_data.index))); ax.set_yticklabels(heatmap_data.index)
for i in range(heatmap_data.shape[0]):
    for j in range(heatmap_data.shape[1]):
        val = heatmap_data.values[i, j]
        if pd.notna(val):
            ax.text(j, i, f"{val:.0f}%", ha="center", va="center", fontsize=8)
fig.colorbar(im, ax=ax, label="Attendance Rate (%)")
ax.set_title("Attendance Rate by Grade & Subject")
plt.tight_layout()
plt.show()
```



7.4 Monthly Attendance Trend

```
In [10]: att_time = attendance.dropna(subset=["Date"]).copy()
att_time["Month"] = att_time["Date"].dt.to_period("M").dt.to_timestamp()
trend = att_time.groupby("Month")["is_present"].mean().mul(100).round(1)

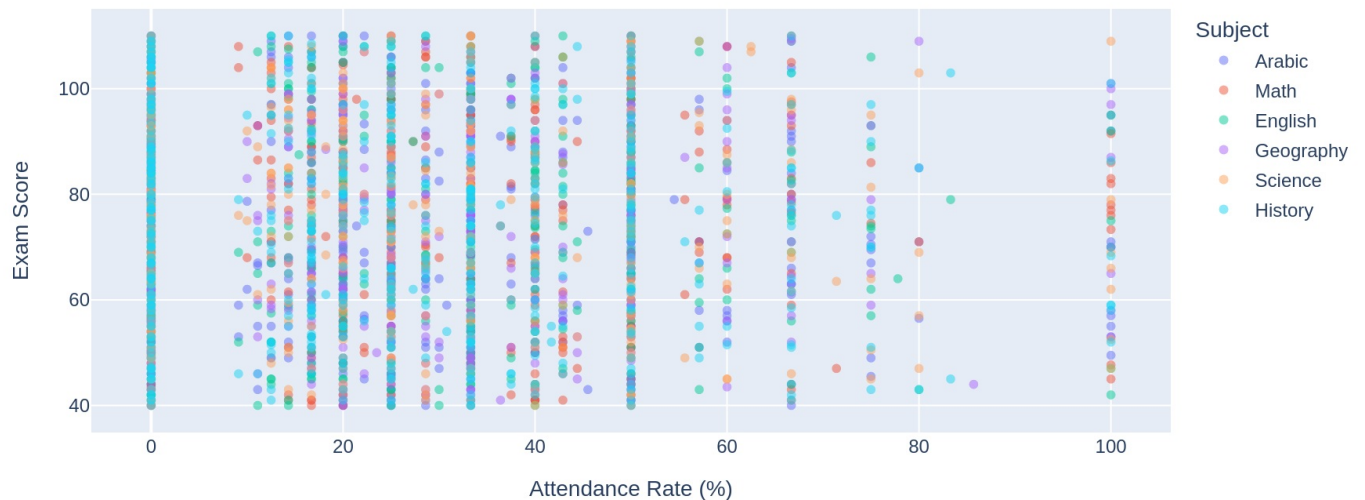
fig, ax = plt.subplots(figsize=(8, 3.5))
ax.plot(trend.index, trend.values, marker="o", color="#4C72B0")
ax.set_ylabel("Attendance Rate (%)")
ax.set_ylim(0, 100)
ax.set_title("Monthly Attendance Trend")
plt.tight_layout()
plt.show()
```



7.5 Attendance vs. Exam Score (Plotly)

```
In [11]: sample = master.dropna(subset=["Attendance_Rate_%", "Exam_Score"]).sample(
n=min(3000, len(master)), random_state=42)
fig = px.scatter(
sample, x="Attendance_Rate_%", y="Exam_Score", color="Subject",
opacity=0.5, title="Attendance Rate vs. Exam Score (by Subject)",
labels={"Attendance_Rate_%": "Attendance Rate (%)", "Exam_Score": "Exam Score"},
)
fig.update_layout(height=450)
fig.show(renderer="png", width=900, height=450, scale=2)
```

Attendance Rate vs. Exam Score (by Subject)



8. Key Insights

- Average exam scores are consistent across subjects (mid-70s), with no subject standing out as substantially harder or easier.
- Correlations between attendance rate and exam score, and between homework completion and exam score, are close to zero across every subject. This dataset does not show the attendance/homework → performance relationship one might expect from real classroom data — consistent with the dataset being synthetically generated rather than collected from real outcomes.
- Attendance rates by grade and subject (heatmap) show no systematic pattern favoring any particular grade or subject.
- The monthly attendance trend is roughly flat over the covered period, without strong seasonal dips (e.g., no clear holiday effect).

Suggested next steps: with real (non-synthetic) data, a natural extension would be a predictive model — e.g., regression

to estimate exam score from attendance/homework features, or classification to flag at-risk students — which was out of scope for this submission.

9. Export Cleaned Dataset

```
In [12]: out_path = Path("../data/cleaned/student_performance_master_cleaned.csv")
out_path.parent.mkdir(parents=True, exist_ok=True)
master.to_csv(out_path, index=False)
print(f"Saved cleaned master table to {out_path} ({master.shape[0]:,} rows)")
```

Saved cleaned master table to ../data/cleaned/student_performance_master_cleaned.csv (28,736 rows)